

SPOTIFY SONG STREAM PREDICTION

USING SUPERVISED LEARNING



TEAM MEMBERS:

160123729006-KRITHIKA REDDY

160123729023-P.YASHICA

160123729024-P.LASYA

What is Supervised learning? ...

SUPERVISED LEARNING IS A TYPE OF MACHINE LEARNING WHERE A MODEL IS TRAINED ON INPUT-OUTPUT PAIRS. THE MODEL LEARNS THE RELATIONSHIP BETWEEN INPUTS (FEATURES) AND THE CORRECT OUTPUTS (LABELS).

KEY CONCEPTS

- INPUT (FEATURES): INFORMATION USED TO MAKE A PREDICTION (E.G, TEMPO, LOUDNESS).
- OUTPUT (LABEL): THE CORRECT ANSWER THE MODEL TRIES TO PREDICT (E.G, NUMBER OF STREAMS).
- TRAINING DATA: DATA THE MODEL LEARNS FROM. IT INCLUDES BOTH INPUTS AND KNOWN OUTPUTS.
- TESTING DATA: NEW DATA THE MODEL HASN'T SEEN BEFORE. USED TO CHECK HOW WELL IT PERFORMS.

About Spotify Song Predictor



THIS CODE IS A MACHINE LEARNING REGRESSION MODEL THAT PREDICTS THE NUMBER OF STREAMS A SPOTIFY SONG WILL GET BASED ON ITS MUSICAL FEATURES, USING A DATASET (SPOTIFY-2023.CSV).

THE MODEL USED IS RANDOM FOREST REGRESSOR, AND IT OUTPUTS HOW CLOSE THE PREDICTIONS ARE TO THE ACTUAL NUMBER OF STREAMS.

WHICH CONTAINS ABOUT 953 ROWS AND 22 COLUMNS WHICH MAKES IT 22872 VALUES IN TOTAL .

FEATURES:

- **DANCEABILITY**
- **TEMPO**
- **ENERGY**
- **POPULARITY**

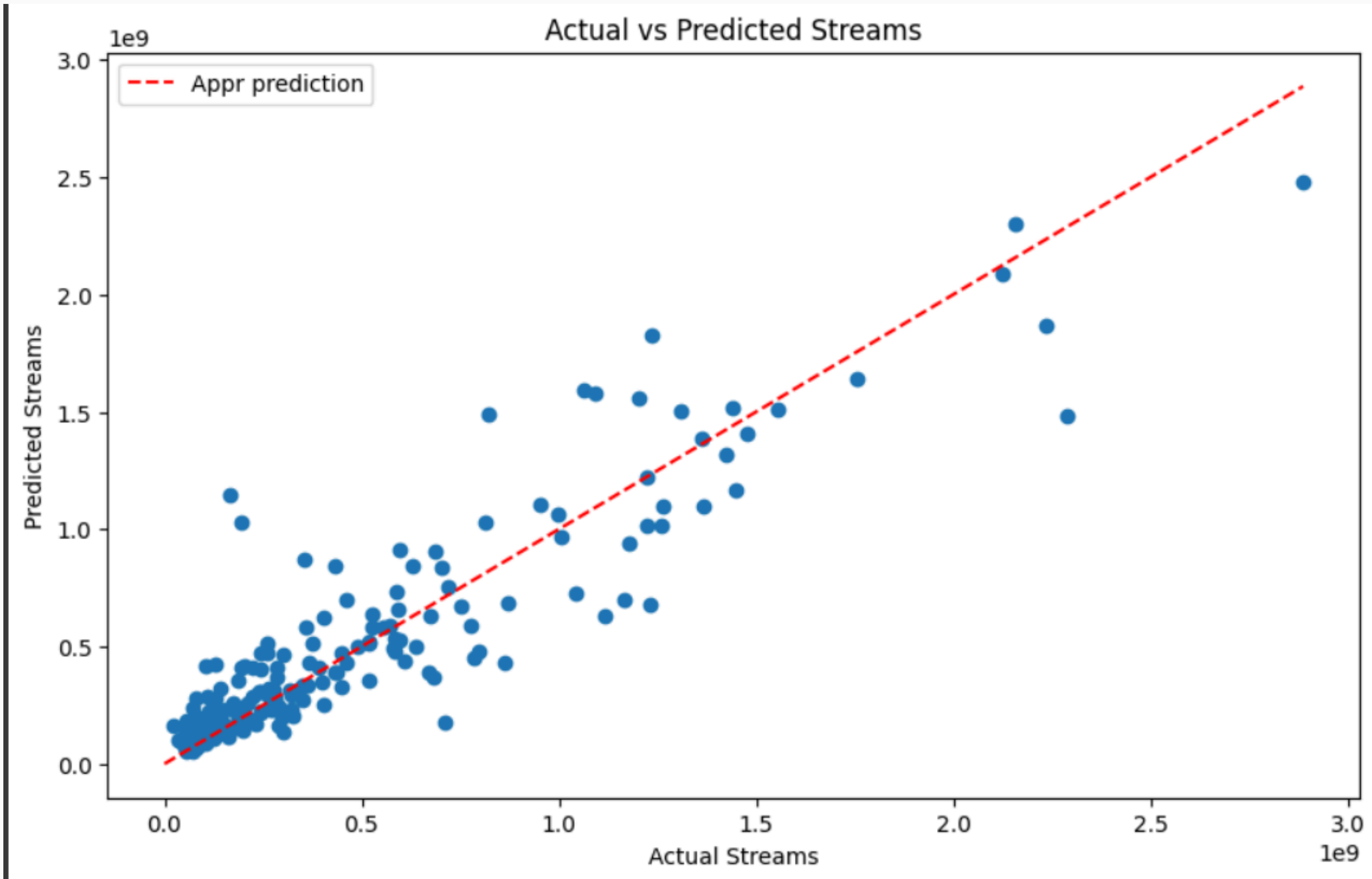
CODE:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
import math
try:
    df = pd.read_csv('/content/spotify-2023.csv', encoding='ISO-8859-1')
    print("Dataset loaded successfully.")
except Exception as e:
    print(f"Error loading dataset: {e}")
    exit()
y_pred = model.predict(x_test_scaled)
mse = mean_squared_error(y_test, y_pred)
rmse = math.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print(f"Mean Squared Error (MSE): {mse:,.0f}")
print(f"Root Mean Squared Error (RMSE): {rmse:,.0f}")
print(f"R-squared: {r2:.3f}")
```

CODE:

```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred)
plt.plot([0, max(y_test)], [0, max(y_test)], '--r', label='Appr prediction')
plt.xlabel('Actual Streams')
plt.ylabel('Predicted Streams')
plt.title('Actual vs Predicted Streams')
plt.legend()
plt.show()
print("\nPrediction Result :")
print("It predicts a song's streams based on its characteristics.")
sample_index = np.random.randint(0, len(X_test))
sample_features = X_test_scaled[sample_index].reshape(1, -1)
actual_streams = y_test.iloc[sample_index]
predicted_streams = model.predict(sample_features)[0]
try:
    sample_track_name = df.loc[y_test.index[sample_index], 'track_name']
    print(f"\nFor the song '{sample_track_name}',")
except KeyError:
    print("\nFor a sample song,")
print(f"Estimated streams: {predicted_streams:,.0f}")
print(f"It guesses based on the song's beat and how danceable it is.")
print(f"(Actual streams: {actual_streams:,.0f})")
```


OUTPUT:



```
Dataset loaded successfully.  
Mean Squared Error (MSE): 44,263,454,023,501,632  
Root Mean Squared Error (RMSE): 210,388,816  
R-squared: 0.819
```

```
Prediction Result :  
It predicts a song's streams based on its characteristics.  
  
For the song 'Thousand Miles',  
Estimated streams: 217,708,508  
It guesses based on the song's beat and how danceable it is.  
(Actual streams: 244,741,137)
```

Conclusion – What We Learned



- How to clean real-world data: We fixed or removed any wrong or missing data to make sure the model works properly.
- How to scale features: We used a method called StandardScaler to put all features on the same scale, which helps the model perform better.
- How to train and test a model: We used a model called Random Forest Regressor to learn from the data and make predictions.
- How to measure performance: We used scores like MSE, RMSE, and R^2 to see how accurate the predictions were.
- How to visualize results: We made a graph comparing the real and predicted stream counts to see how well the model did.

THANK YOU!!!

